

API Documentation of AlumniConnect

1. Authentication & Core Users Module (/api/users)

This module handles identity, role management, and system access.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/users/register	Creates a new user account.	Body: • email (String, required) • password (String, required) • role (String, enum: 'admin', 'alumni', 'student')
POST	/api/users/login	Authenticates user and returns JWT.	Body: • email (String, required)• password (String, required)
GET	/api/users	Admin route to fetch all users.	Query Params: • role (String)• AccountStatus (String)• page (Number)• limit (Number)
GET	/api/users/:id	Retrieves a specific core user document.	Params: id (ObjectId)
PATCH	/api/users/:id/status	Updates account verification status.	Body: • AccountStatus (String, enum: 'unverified', 'verified', 'suspended')

DELETE	/api/users/:id	Hard deletes a user from the system.	None
---------------	----------------	--------------------------------------	------

2. Profile Module (/api/profiles)

This module manages the heavy biographical and professional data.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/profiles	Creates the public profile linked to a user.	Body: • userId (ObjectId, required) • firstName (String, required) • lastName (String, required) • professionalHeadline (String, optional) • skills (Array of Strings, optional) • location (Array of Strings, optional) • profilePicture (String, optional) • education (Array of Objects): institution (String), degree (String), fieldOfStudy (String), graduationYear (Number) • experience (Array of Objects): company (String), position (String), startDate (Date), endDate (Date), isCurrent (Boolean)

GET	/api/profiles	Fetches the searchable alumni directory.	Query Params: • skills (String)• location (String)• graduationYear (Number)• company (String)• page (Number)• limit (Number)
GET	/api/profiles/:userId	Retrieves a specific profile by User ID.	Params: userId (ObjectId)
PUT	/api/profiles/:userId	Completely updates a user's profile.	Body: Full profile object identical to the POST payload.
DELETE	/api/profiles/:userId	Deletes the profile document.	None

3. Mentorship Module (/api/mentorships)

This module handles the lifecycle of many-to-many mentoring relationships.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/mentorships	Initiates a new mentorship connection.	Body: • mentorId (ObjectId, required) • menteeId (ObjectId, required) • Initialmessage (String, optional, max 500 chars)
GET	/api/mentorships	Retrieves mentorship connections.	Query Params: • mentorId (ObjectId)• menteeId (ObjectId)• status

			(String: 'pending', 'active', 'completed', 'cancelled') • page (Number) • limit (Number)
GET	/api/mentorships/:id	Retrieves a specific mentorship document.	Params: id (ObjectId)
PATCH	/api/mentorships/:id	Updates the status and tracks lifecycle dates.	Body: • status (String, enum: 'pending', 'active', 'completed', 'cancelled') • mentorshipStartedAt (Date, optional) • mentorshipEndedAt (Date, optional)
DELETE	/api/mentorships/:id	Permanently deletes a mentorship record.	None

4. Job Referrals Module (/api/jobs)

This module manages ephemeral career opportunities posted by users.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/jobs	Creates a new job posting on the board.	Body: • postedBy (ObjectId, required) • company (String, required) • jobTitle (String, required) • jobDescription (String, required) • requirements

			(String, required) • applyLink (String, required) • isActive (Boolean, optional, default: true) • expiresAt (Date, required)
GET	/api/jobs	Fetches active jobs for the board.	Query Params: • company (String)• jobTitle (String)• isActive (Boolean)• page (Number)• limit (Number)
GET	/api/jobs/:id	Retrieves a specific job posting.	Params: id (ObjectId)
PUT	/api/jobs/:id	Fully updates an existing job posting.	Body: Full job object identical to the POST payload.
PATCH	/api/jobs/:id	Manually toggles visibility without deleting.	Body: • isActive (Boolean)
DELETE	/api/jobs/:id	Hard deletes a job before the TTL expires.	None

5. Events Module (/api/events)

This manages the core event details scheduled by organizers.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/events	Creates a new scheduled gathering.	Body: • organizer (ObjectId, required) • title (String,

			required) • description (String, required) • eventDate (Date, required) • location (String, required) • isVirtual (Boolean, optional, default: false)
GET	/api/events	Fetches a list of upcoming events.	Query Params: • isVirtual (Boolean)• startDate (Date) • endDate (Date)• page (Number)• limit (Number)
GET	/api/events/:id	Retrieves full details of a single event.	Params: id (ObjectId)
PUT	/api/events/:id	Fully updates the event itinerary/details.	Body: Full event object identical to the POST payload.
DELETE	/api/events/:id	Deletes the event (Must trigger cascading RSVP deletion).	None

6. Event RSVP Module (/api/events/:eventId/rsvps)

This acts as the junction API to map attendees to events.

Method	Endpoint	Purpose	Request Payload / Query Params
PUT	/api/events/:eventId/rsvps	Creates or updates a user's RSVP to an event.	Body: • userId (ObjectId, required) • rsvpStatus (String, required, enum:

			'attending', 'not attending', 'maybe')
GET	/api/events/:eventId/ rsvps	Retrieves the attendee list for a specific event.	Query Params: • rsvpStatus (String)• page (Number)• limit (Number)
DELETE	/api/events/:eventId/ rsvps/:userId	Completely removes a user's RSVP record.	None

7. Messaging Module (/api/conversations & /api/messages)

These routes handle the RESTful metadata and history fetching for the chat system.

Method	Endpoint	Purpose	Request Payload / Query Params
POST	/api/conversations	Creates a new chat thread.	Body: • participants (Array of ObjectIds, required)
GET	/api/conversations	Retrieves the inbox view for the logged-in user.	Query Params: • userId (ObjectId)• page (Number)• limit (Number)
PATCH	/api/conversations/:id	Updates the thread preview upon new messages.	Body: • lastMessagePreview (String, optional) • lastMessageAt (Date, optional)
DELETE	/api/conversations/:id	Deletes a conversation thread.	None
POST	/api/messages	Saves an individual	

		chat bubble to the database.	Body: • conversationId (ObjectId, required) • senderId (ObjectId, required) • message (String, required) • isRead (Boolean, optional, default: false)
GET	/api/messages/:conversationId	Retrieves the message history for a specific thread.	Query Params: • page (Number) • limit (Number)
PATCH	/api/messages/:id/read	Updates the read receipt for a specific message.	Body: • isRead (Boolean, required)
DELETE	/api/messages/:id	Deletes an individual message (unsend).	None